

deepin

deepin-liferaft

内存高压下的桌面守护：先冻结，再询问，可恢复

deepin 社区开源项目 · github.com/st0nie/deepin-liferaft

以用为本 拥抱创新

www.deepin.org

Content

目录

01

项目概述

macOS 式压力对话框

02

问题背景

内存耗尽，桌面先死

03

工作机制

每秒一次判定循环

04

安全设计

冻结可逆不越界

05

部署使用

上手指南与服务管理

06

资源与验证

13 MiB 常驻代价

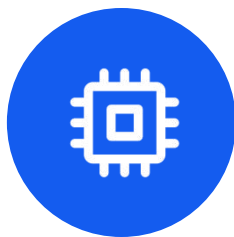
01

项目概述

为 Deepin 带回 macOS 式的内存高压体验



三大设计支柱



macOS 式交互

弹窗列出内存占用最高的应用，可一键恢复或强制退出；最终处置权交给用户，未保存的工作不会被无声吞掉。



oomd 检测策略

沿用 Fedora Workstation 的 systemd-oomd 判据：full avg10 超过 50% 持续 20 秒，且最近 30 秒存在内存回收活动。



cgroup 原生冻结

基于 cgroup v2 的 freeze / kill 原语，以应用为单位暂停资源消耗；进程状态完整保留，解冻即恢复原样。

02

问题背景

内存耗尽时，桌面先于系统
死去



既有方案的三块拼图



● 内核 OOM Killer: 兜底但太晚

触发条件是分配失败且回收无效；交换区反复换入换出可让卡顿持续数分钟而 OOM 始终不触发，被杀进程未必是肇事者，数据无从保存。



● 用户态主动处置：提前但只会杀

oomd、earlyoom、nohang、systemd-oomd 在系统变慢但尚可行动的窗口介入，处置清一色是 SIGKILL——无中间态、无用户参与，可能误杀正在使用的前台应用。



● macOS 的第三条路：先暂停，再问用户

弹窗前系统已自动暂停占用最高的应用：窗口还在、进程挂起；用户决定恢复或强制退出，把数据保全的机会留给用户。

03

工作机制

每秒一次，把卡顿翻译成可度量的信号



触发与处置流水线



04

安全设计

冻结是能力，不失控才是底线



冻结的四条不变量

宁可放弃判定，也绝不误伤、绝不遗留。



只冻自己冻的

已被他人冻结的组，不接管、不解冻



绝不冻自身

冻结前排除 liferaft 所在 cgroup



读失败不触发

采样读取失败即放弃本次判定



退出前必解冻

关窗、SIGTERM 均先解冻全部持有组

05

部署使用

从安装到实测弹窗，五分钟
上手



三步启用



● 第一步：安装并启用服务

安装 deb 包后执行 `systemctl --user enable deepin-liferaft.service --now`，服务即以隐藏模式常驻，弹窗随压力自动出现。



● 第二步：开启内核 PSI

部分 Deepin 内核默认禁用 PSI：在 `/etc/default/grub` 的 `GRUB_CMDLINE_LINUX` 行尾追加 `psi=1`，执行 `sudo update-grub` 并重启生效；deb 包安装会自动追加。



● 第三步：验证与实测

`cat /proc/pressure/memory` 确认 `some/full` 指标存在；运行 `memory_eater.py` 制造高压，看到弹窗即为工作正常。

PSI 开启与验证

```
> cat /etc/default/grub
# Generated by deepin-installer
GRUB_BACKGROUND="/boot/grub/themes/deepin/background.jpg"
GRUB_CMDLINE_LINUX_DEFAULT="video=efifb:nobgrt splash quiet loglevel=0 locales=zh_CN.UTF-8"
GRUB_DEFAULT=0
GRUB_DISTRIBUTOR=" /usr/bin/lsb_release -d -s 2>/dev/null"
GRUB_THEME="/boot/grub/themes/deepin/theme.txt"
GRUB_TIMEOUT=1
GRUB_GFXMODE=auto
DEEPIN_GFXMODE_DETECT=1

GRUB_CMDLINE_LINUX="zswap.enabled=1 zswap.shrinker_enabled=1 zswap.compressor=lz4 zswap.max_pool_percent=0 psi=1"
~Desktop
>
```

追加内核参数

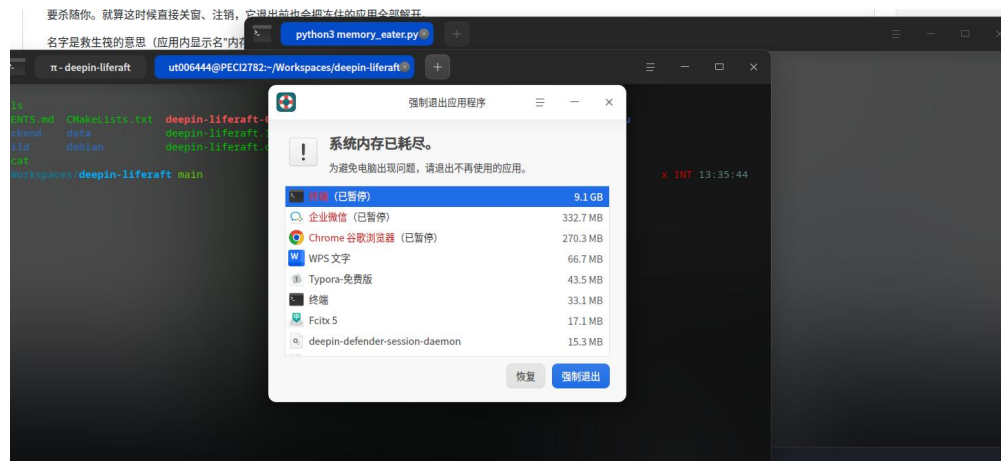
在 `/etc/default/grub` 行尾追加 `psi=1`, `update-grub` 重启生效; deb 安装可自动完成。

```
> cat /proc/pressure/memory
some avg10=0.00 avg60=0.00 avg300=0.00 total=16020978
full avg10=0.00 avg60=0.00 avg300=0.00 total=15743550
~
>
```

确认压力指标可用

`cat /proc/pressure/memory` 可见 `some/full` 与三档均值, 压力信号就绪。

实测效果



高压来临，弹窗兜底

memory_eater 吃满内存即触发「系统内存已耗尽」对话框；已暂停的应用以红字标出，占用一目了然——图中终端以 9.1 GB 居首，企业微信与 Chrome 随后。

恢复还是强杀，由你决定

选中应用点「恢复」立即解冻继续工作；点「强制退出」整组终止；直接关闭对话框也会解冻全部已暂停应用，系统马上恢复。

白名单与服务管理



白名单三级合并

包默认 /usr/share、管理员 /etc、用户 ~/.config 三级白名单合并为同一集合；名单内的应用永不进入对话框、永不冻结。修改后 `systemctl --user restart` 生效，启动日志记录加载条数。



服务与内核参数

systemd 用户服务以 `--hidden` 常驻；deb 包自动为禁用 PSI 的内核追加 `psi=1` 参数并 `update-grub`；`journalctl --user` 可查每次触发、冻结与查杀记录。

06

资源与验证

以约 13 MiB 的常驻代价换取
桌面的可控



隐藏模式实测数据



PSS 常驻 (MiB)

启动 3 秒后实测，优化前为 19.4 MiB；
同机空 DApplication 的 RSS 约 41.6
MiB，可见大头是 DTK/Qt 共享库。



RSS 表观 (MiB)

优化前 60.7 MiB；延迟构建控件与图
标数据、压力低时不扫描应用 cgroup，
是两项主要优化手段。



私有脏页 (MiB)

优化前 9.85 MiB；ctest 自测覆盖 PSI
解析、阈值边界、freezer 往返与白
名单解析，行为可回归。

Thanks For Watching

深度科技社区



微信公众号

